esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orientée Objet & Programmation JAVA

Chapitre 8: Les Collections

Objectifs du chapitre

- Comprendre et appliquer le concept d'interfaces
- Maîtriser la collection `List`, comprendre son architecture et ses principales méthodes
- Exploiter la classe utilitaire `Collections`
- Distinguer les interfaces `Comparable` et `Comparator`

Les interfaces

Les interfaces

- En Java, une interface est un ensemble de méthodes abstraites (sans corps) et de constantes, qui peut être implémentée par n'importe quelle classe.

- Elle définit un ensemble de fonctionnalités qu'une classe peut offrir sans définir comment ces fonctionnalités sont implémentées.

- En d'autres termes, une interface peut être considérée comme un contrat que toute classe qui l'implémente doit respecter.

Les interfaces

En Java, une interface est déclarée à l'aide du mot-clé **interface** suivi du nom choisi pour l'interface.

Une classe **implémente** ensuite une ou plusieurs **interfaces** afin de **définir** le comportement spécifié par celles-ci.

```
interface MyInterface{
    float PI = 3.14f;
    void meth1();
    int meth2(String str);
}
```

Les interfaces

- Toutes les méthodes sont implicitement **abstraites** et doivent être implémentées par les classes qui la réalisent
- Tous les membres (méthodes et attributs) sont **publics** par défaut.
- Tout attribut déclaré est automatiquement **constant** (final) et **statique** (static), ce qui signifie qu'il appartient à l'interface et ne peut pas être modifié.

```
interface MyInterface{
    float PI = 3.14f; //static, final et public
        sont optionnels

    void meth1(); //abstract et public sont
        optionnels

    int meth2(String str);
}
```

NB : Le niveau de visibilité des membres d'une interface ne peut pas être modifié

Les interfaces

Une classe qui implémente une interface doit **fournir une implémentation complète (le corps)** de toutes les méthodes déclarées dans cette interface.

Si la classe **n'implémente pas toutes les méthodes**, elle doit alors être déclarée **comme abstraite**, car elle ne satisfait pas entièrement le **contrat** défini par l'interface.

```
class MyClass implements MyInterface{
    void meth1(){
        //Code d'implémentation de la méthode 1
        }
    int meth2(String str){
        //Code d'implémentation de la méthode 2
        }
}
```

Les interfaces

Si une classe contient que **des méthodes abstraites**, on peut la **convertir en une interface.**

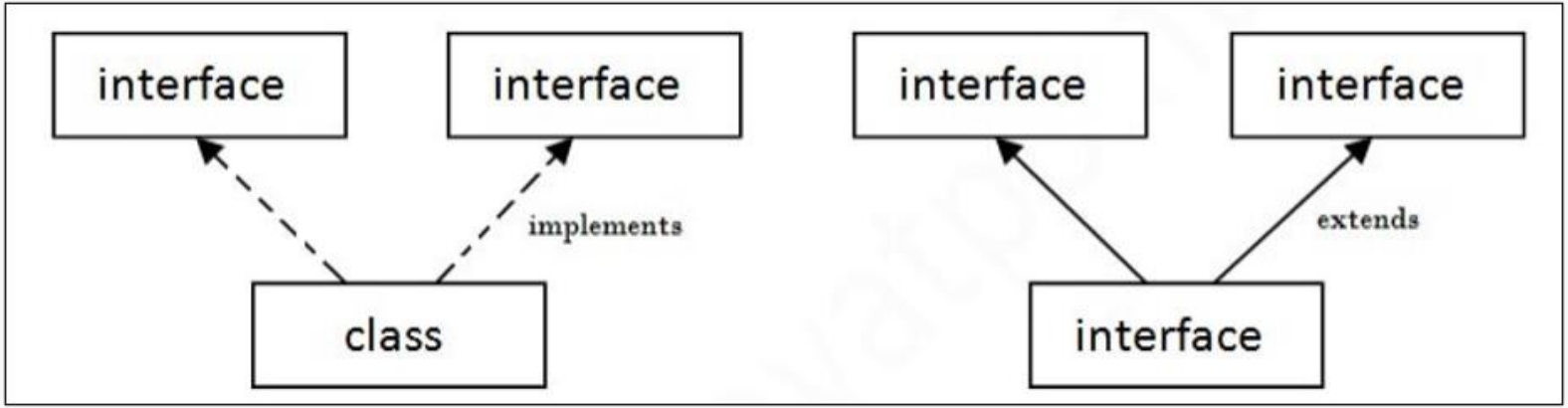
```
abstract class MyClass{

    public abstract void meth1();
    public abstract int meth2(String str);
    public abstract float meth3(int x);
}
```

```
interface MyInterface{
    void meth1();
    int meth2(String str);
    float meth3(int x);
}
```

Les interfaces

Une classe peut implémenter **plusieurs** interfaces à la fois, et une interface peut hériter de **plusieurs** autres interfaces à la fois.

NB : En JAVA l'héritage multiple pour une classe n'est pas permis, donc les interfaces répondent à cette problématique.



Les méthodes par défaut

- Grâce aux méthodes par **défaut (default methods)**, une interface peut désormais **fournir une implémentation commune pour certaines méthodes.**
- Cela permet de **réduire la duplication** de code et **d'ajouter de nouvelles fonctionnalités aux interfaces** sans rompre la compatibilité avec les classes existantes

```
interface Math1{
    int add(int x,int y);
    int multiply(int x,int y);
    default int divide(int x,int y){
        return x / y;
    }
}
```

Les méthodes par défaut

Une méthode par défaut **peut être redéfinie**, mais ce n'est **pas obligatoire** puisqu'elle n'est **pas abstraite**.

```
class MyClass implements Math1{
    public int add(int x,int y){
        return x + y;}
    public int multiply(int x,int y){
        return x * y;}
    public int divide(int x,int y){
         int z = Math1.super.divide(x,y);
         return z / 2;
     }
}
```

Les méthodes statiques

Bien que ce code **ne contienne aucune erreur de compilation**, une **exception sera levée à l'exécution**, car la **division par zéro est interdite** en Java et provoque une **ArithmeticException**.

```
public class TestException{
    public static void main(String[] args) {
        int x = 5;
        int y = 0;
        System.out.println("Avant incident");
        System.out.println("La division de x/y
            est :" + x / y);
        System.out.println("Après incident");
    }
}
```

Les interfaces génériques

- Une **interface générique** est une interface paramétrée par un ou plusieurs types, ce qui permet de créer **des classes et des méthodes flexibles et réutilisables** avec différents types de données.
- Pour passer un paramètre à une interface, nous spécifions un seul **caractère majuscule** pour le nom du paramètre de type générique **("E", "T", "K", etc.).**

```
interface Math1<T>{
    T add(T x,T y);
    T multiply(T x,T y);
  }
```

```
interface Math2<T,K>{
    K add(T x,T y);
    K multiply(T x,T y);
    }
```

Les interfaces génériques

Cela permet d'avoir différentes implémentations des mêmes méthodes avec un type de données différent pour chaque classe.

```
class IntegerMath implements Math1<Integer>{
 public Integer add(Integer x,Integer y){ return x + y; }
 public Integer multiply(Integer x,Integer y){ return x * y; }
}
```

```
class FloatMath implements Math1<Float>{
    public Float add(Float x,Float y){ return x + y; }
    public Float multiply(Float x,Float y){ return x * y; }
}
```

NB: Seuls les types de Référence peuvent être transmis à une interface.

Enum

En Java, **les énumérations (enum)** sont des types de “classes” spéciales qui permettent **de définir un ensemble fixe de constantes ou de valeurs énumérées.**

Les **enums** sont souvent utilisés pour définir des types de données avec un nombre limité de choix, comme **les jours de la semaine**, **les mois de l'année, les types de véhicules, etc.**

Les **enums** sont déclarés en utilisant le mot-clé enum, suivi d'une liste de constantes (valeurs énumérées) entre accolades.

Les Collections

Collection

En Java, une **collection** est une structure de données permettant de **regrouper** et **gérer plusieurs objets de manière efficace**.

Les collections sont principalement utilisées pour :

- Stocker, rechercher et manipuler efficacement des données.
- Transmettre des groupes d’objets d’une méthode à une autre, facilitant ainsi la gestion et le partage des informations dans un programme.

Collection

En Java, **les collections** constituent un ensemble de **classes et d'interfaces** offrant une architecture unifiée pour **gérer, structurer et traiter efficacement des groupes d'objets.**

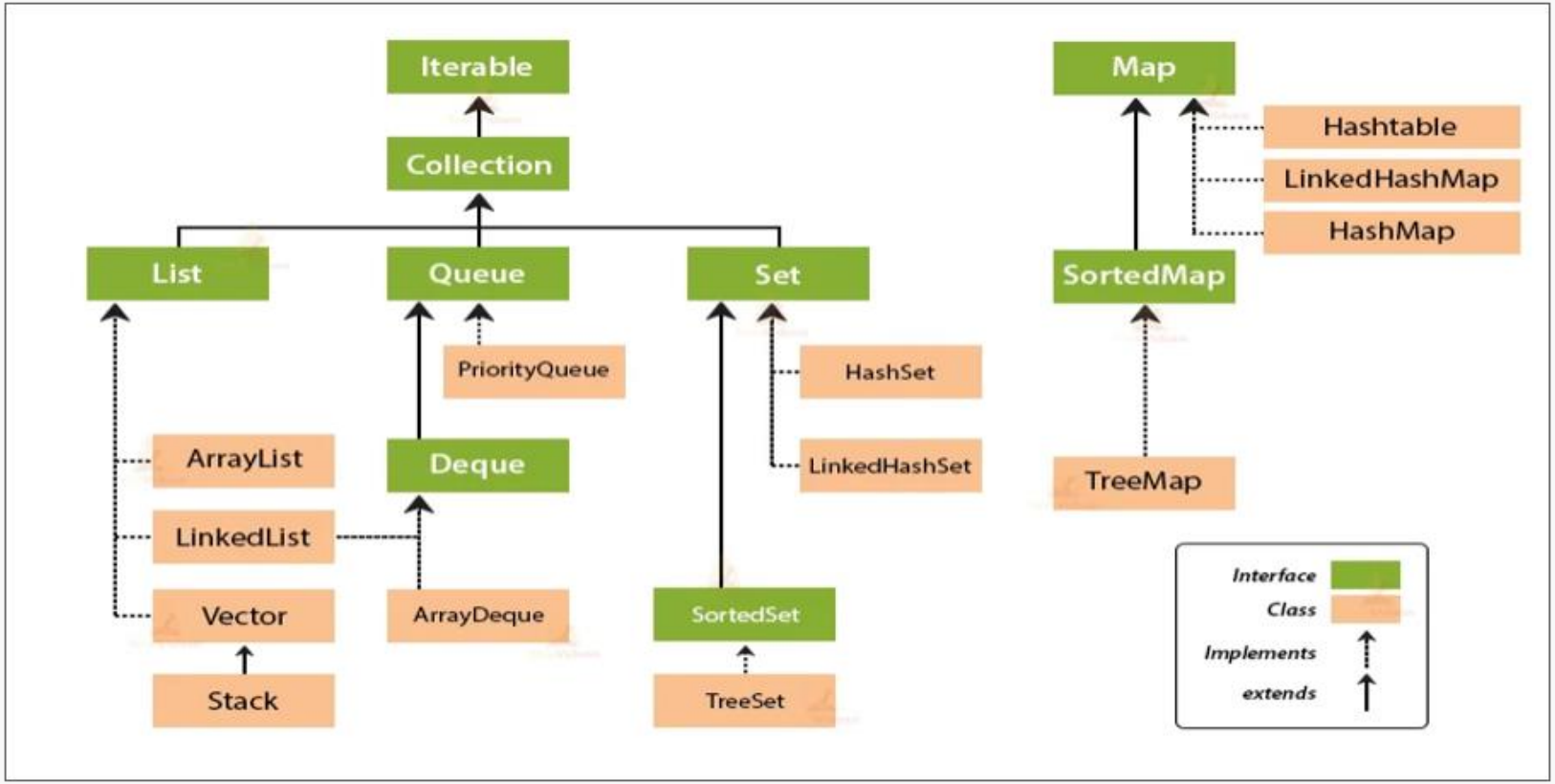
Il existe divers types **d'implémentations**, chacune adaptée à des besoins spécifiques :

- ✓ Certaines autorisent **les doublons, d'autres les interdisent**
- ✓ Certaines préservent **l'ordre d'insertion, d'autres sont non ordonnées**
- ✓ Certaines imposent des **contraintes particulières**, comme le type des éléments ou l'interdiction des valeurs nulles.

Collection VS Tableaux

	Collection	Tableau
Taille	Taille dynamique	Taille Fixe
Type d'element	Objet (pas de types primitifs)	Types primitifs et objet
Accès	Accès sequentiel	Accès par index
Manipulation	Des méthodes prédéfinies pour l'ajout , la suppression …	Créer des méthodes pour l'ajout, la suppression, …
Performance	Plus performants en terme de recherche ou le tri d'éléments.	Plus performants en terme d'accès et manipulation

Architecture des Collections



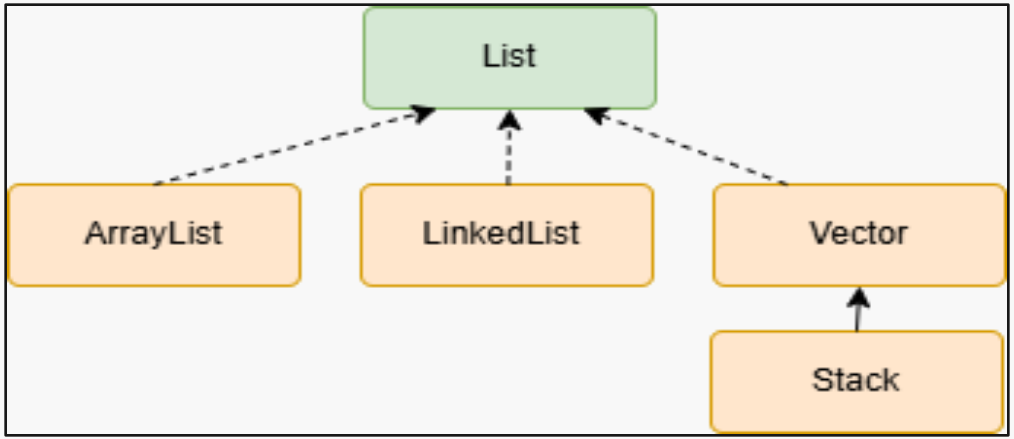
Architecture des Collections

Les collections en Java reposent sur une hiérarchie d'interfaces et de classes :

- **Collection** et **Map** sont les interfaces principales.
- Leurs sous-interfaces (**List, Set,Queue**) définissent différents types de structures de données.
- Chaque **interface** dispose **d'implémentations** concrètes (comme **ArrayList,HashSet, TreeMap**) qui diffèrent par leur ordre, la gestion des doublons et leurs performances.

List

Interface List



List est une **interface** qui permet de stocker des éléments ordonnés.

Chaque élément est accessible par sa position dans la liste, à l'aide d'**un indice entier** qui commence à **0** pour le premier élément.

Les classes qui implémente l’interface **List** sont des **"tableaux" extensibles à volonté.**

ArrayList VS Vector

Méthode	Explication	Type de retour	Paramètres
add(E e)	Ajoute un élément à la fin de la collection.	boolean	e : l'élément à ajouter.
addAll(Collection<? extends E> c)	Ajoute tous les éléments de la collection spécifiée à la fin de cette collection.	boolean	c : la collection à ajouter.
clear()	Supprime tous les éléments de la collection.	void	N/A
contains(Object o)	Retourne true si la collection contient l'élément spécifié, sinon false.	boolean	o : l'élément à rechercher.
containsAll(Collection<?> c)	Retourne true si cette collection contient tous les éléments de la collection spécifiée, sinon false.	boolean	c : la collection à rechercher.
isEmpty()	Retourne true si la collection ne contient aucun élément, sinon false.	boolean	N/A
iterator()	Retourne un itérateur sur les éléments de la collection.	Iterator<E>	N/A

ArrayList

```
List monArrayList = new ArrayList(10);
```

Instanciation d'un Arraylist de reference List

Remarque: L'attribution de la taille d'un tableau dynamique est facultatif !

ArrayList

- Un **ArrayList** est un “tableau” qui se redimensionne automatiquement. Il accepte tout **type d'objets, null y compris.**
- Chaque instance **d'ArrayList** a une capacité, qui définit le nombre d'éléments qu'on peut y stocker.
- Au fur et à mesure qu'on ajoute des éléments et qu'on "dépasse" la capacité, **la taille augmente en conséquence.**

ArrayList

```java
import java.util.List;
import java.util.ArrayList;
public class ListeExemple {
    public static void main(String[] args) {
        // Création d'une liste générique qui accepte tous type de données
        List maListe = new ArrayList<>();
        maListe.add("Bonjour");
        maListe.add(8);
        maListe.add(true);
        maListe.add(null);
        // Une liste peut contenir des éléments redondants
        maListe.add("Bonjour");
        System.out.println(maListe);
    }
}
```

ArrayList

```java
import java.util.List;
import java.util.ArrayList;
public class ListeExemple {
    public static void main(String[] args) {
        // Création d'une liste de chaînes de caractères de taille 2
            List<String> maListe = new ArrayList<>(2);
        // Ajout de 3 éléments à la liste , la liste se redimensionne
            maListe.add("Bonjour");
            maListe.add("à");
            maListe.add("tous");
        // Accès à un élément de la liste par son index
            String deuxiemeElement = maListe.get(1);
            System.out.println("Deuxième élément : " + deuxiemeElement);
        // Modification d'un élément de la liste
            maListe.set(2, "monde");
            System.out.println("Liste modifiée : " + maListe);
    }
}
```

ArrayList

```java
public class ListeExemple {
    public static void main(String[] args) {
        // Création d'une liste de chaînes de caractères de taille 2
            List<String> maListe = new ArrayList<>(2);
        // Ajout de 3 éléments à la liste , la liste se redimensionne
            maListe.add("Bonjour");
            maListe.add("à");
            maListe.add("tous");
        // Suppression du premier élément de la liste
            maListe.remove(0);
            System.out.println("Liste modifiée : " + maListe);
        // Recherche d'un élément dans la liste
            System.out.println(maListe.contains("Bonjour"));
    }
}
```

Pour afficher les éléments d'une liste avec l'instruction "System.out.println()" la redéfinition de la méthode toString() est nécessaire.

ArrayList

```java
public class ListeExemple {
    public static void main(String[] args) {
        List<String> maListe = new ArrayList<>();
        maListe.add("Bonjour");
        maListe.add("à");
        maListe.add("tous");
   // Suppression du premier élément de la liste
        maListe.remove(0);
   // Recherche d'un élément dans la liste
        System.out.println(maListe.contains("Bonjour"));
    }
}
```

Pour rechercher ou supprimer un élément , la redéfinition de la méthode equals(Object obj) est nécessaire pour comparer deux objets selon leurs attributs.

Collection vs Collections

-**Collection** est une **interface** qui définit le comportement générique des collections de données en Java.

-**Collections** est une **classe** utilitaire qui possède des méthodes statiques pour manipuler des collections. Cette classe contient des méthodes qui permettent de **trier, rechercher, inverser, mélanger, etc.** les éléments d’une collection.

La classe Collections

Méthode	Description
copy(List<? super T> dest, List<? extends T> src)	Copie tous les éléments de la liste source vers la liste de destination
fill(List<? super T> list, T obj)	Remplit tous les éléments de la liste avec l'objet spécifié
max(Collection<? extends T> coll)	Retourne l'élément maximum de la collection donnée, selon l'ordre naturel des éléments
min(Collection<? extends T> coll)	Retourne l'élément minimum de la collection donnée, selon l'ordre naturel des éléments

La classe Collections

Méthode	Description
reverse(List<?> list)	Inverse l'ordre des éléments de la liste
shuffle(List<?> list)	Mélange les éléments de la liste dans un ordre aléatoire
sort(List<T> list)	Trie les éléments de la liste dans l'ordre naturel des éléments
swap(List<?> list, int i, int j)	Échange les éléments à la position i et j dans la liste

La classe Collections

- La méthode **Collections.sort(List l)** est une méthode statique fournie par la classe Collections, qui permet de trier les éléments d'une collection donnée dans l'ordre naturel des éléments.

--> Si la méthode **sort()** est utilisée, il faut obligatoirement que les éléments inclus dans la liste implémentent tous l'interface **Comparable** sinon une exception de type ***ClassCastException*** est levée.

L’interface Comparable<T>

L’interface **Comparable** est utilisée pour permettre la comparaison d'objets entre eux. Cette interface définit une méthode **compareTo** qui prend en argument un **objet** et retourne un **entier**.

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

L’interface Comparable<T>

La méthode **compareTo** compare l'objet courant avec l'objet donné en argument et renvoie :

- un nombre négatif si l'objet courant est inférieur à l'objet donné en argument.
- zéro si l'objet courant est égal à l'objet donné en argument.
- un nombre positif si l'objet courant est supérieur à l'objet donné en argument.

  - a.compareTo (b) == 0 si a.equals To(b)
  - a.compareTo (b) < 0 si a plus « petit »que b
  - a.compareTo (b) > 0 si a plus « grand » que b

L’interface Comparable<T>

La méthode **compareTo** compare l'objet courant avec l'objet donné en argument et renvoie :

- un nombre négatif si l'objet courant est inférieur à l'objet donné en argument.
- zéro si l'objet courant est égal à l'objet donné en argument.
- un nombre positif si l'objet courant est supérieur à l'objet donné en argument.

  - a.compareTo (b) == 0 si a.equals To(b)
  - a.compareTo (b) < 0 si a plus « petit »que b
  - a.compareTo (b) > 0 si a plus « grand » que b

L’interface Comparator

La méthode **statique** `Collections.sort(List l, Comparator c)` permet de **trier les éléments d’une liste** selon des **critères personnalisés** définis par un **comparator**.

Elle offre la possibilité de réaliser des **tris multi-critères**.
Par exemple, il est possible de **trier une liste de personnes** d’abord par **âge**, puis – en cas d’égalité – par **nom**, pour obtenir un **classement plus précis et cohérent**.

L'interface Comparator

```
public interface Comparator<T> {
    int compare(T o1, T o2);
}
```

int compare(T o1, T o2) qui doit renvoyer

- un entier positif si **o1 est « plus grand » que o2**
- 0 si **o1 a la même valeur (au sens de equals)** que **o2**
- un entier négatif si **o1 est « plus petit » que o2**

Comparable vs Comparator

	Comparable	Comparator
Séquence de tri	Comparable fournit une séquence de tri unique. En d'autres termes, nous pouvons trier une collection sur la base d'un seul élément, tel que l'identifiant, le nom et l'adresse.	Comparator fournit plusieurs séquences de tri. En d'autres termes, nous pouvons trier la collection en fonction de plusieurs éléments tels que l'identifiant, le nom, l'adresse, etc.
La méthode	Comparable fournit la méthode **compareTo()** pour trier les éléments.	Comparator fournit la méthode **compare()** pour trier les éléments.
Package	Comparable existe dans le package java.lang.	Comparator existe dans le package java.util.
Trier	Nous pouvons trier les éléments d'une liste de type Comparable avec la méthode `Collections.sort(List)`.	Nous pouvons trier les éléments d'une liste de type Comparator avec la méthode `Collections.sort(List, Comparator)`

Merci pour votre attention